



Tribhuvan University

Faculty of Humanities and Social Sciences

**A LAB REPORT
Of
INFORMATION SECURITY (CACS 459)**

Submitted to:

Department of Computer Application

Mechi Multiple Campus

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by:

Pawan Adhikari (Roll no.35)

(Ashoj 2081)

Under the Supervision of

Krishna Pd. Acharya

Internal Examiner

External Examiner

Lab Report for Information Security

Lab 1: Implementation of Caesar Cipher

- i) Introduction/Concept
- ii) Program code
- iii) Output

Lab 2: Implementation of Vigenere Cipher

- i) Introduction/Concept
- ii) Program code
- iii) Output

Lab 3: Implementation of Playfair Cipher

- i) Introduction/Concept
- ii) Program code
- iii) Output

Lab 4: Implementation of Rail Fence Ciphers

- i) Introduction/Concept
- ii) Program code
- iii) Output

Lab 5: Implementation of password based authentication system

- i) Introduction/Concept
- ii) Program code
- iii) Output

Lab 1: Caesar Cipher implementation

Introduction:

Caesar Cipher is the earliest known, and the simplest, use of a substitution cipher was by Julius Caesar. The Caesar cipher involves replacing each letter of the alphabet with the letter standing three places further down the alphabet. The alphabet is wrapped around, so that the letter following Z is A. We can define the transformation by listing all possibilities, as follows:

Plaintext: a b c d e f g h i j k l m n o p q r s t u v w x y z

cipher: D E F G H I J K L M N O P Q R S T U V W X Y Z A B C

Source Code:

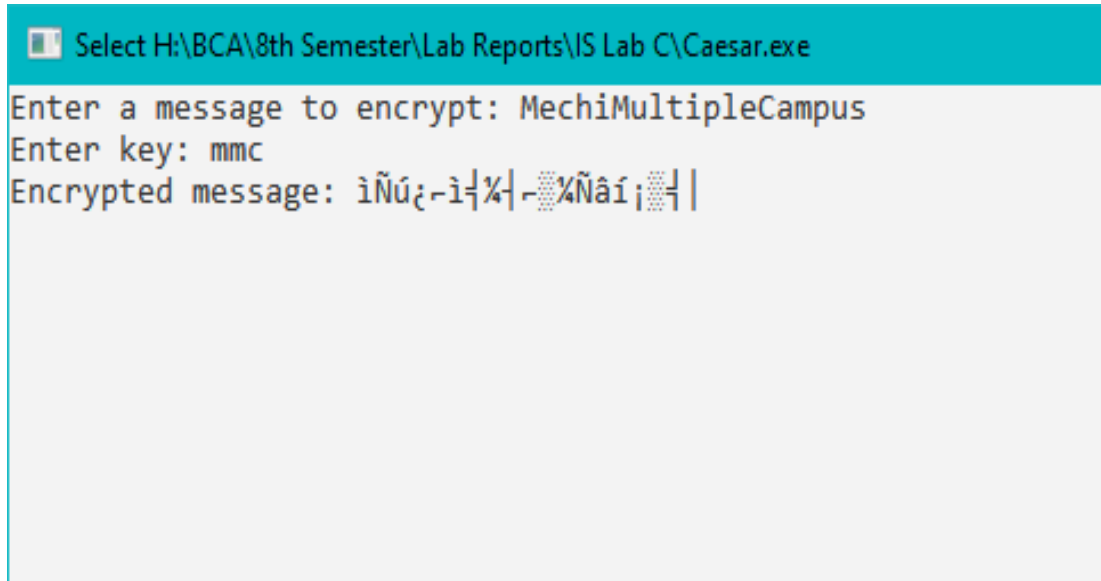
```
#include<stdio.h>
#include<conio.h>
int main()
{
    char message[100], ch;
    int i, key;
    clrscr();
    printf("Enter a message to encrypt: ");
    gets(message);
    printf("Enter key: ");
    scanf("%d", &key);
    for(i = 0; message[i] != '\0'; ++i)
    {
        ch = message[i];
        if(ch >= 'a' && ch <= 'z')
        {
            ch = ch + key;
            if(ch > 'z')
            {
                ch = ch - 'z' + 'a' - 1;
            }
            message[i] = ch;
        }
        else if(ch >= 'A' && ch <= 'Z')
        {
            ch = ch + key;
            if(ch > 'Z')
            {
                ch = ch - 'Z' + 'A' - 1;
            }
            message[i] = ch;
        }
    }
}
```

```

printf("Encrypted message: %s", message);
getch();
return (0);
}

```

Output:



Lab 2: Implementation of Vigenere Cipher

Introduction:

The Vigenere cipher is a classical encryption technique that was developed in the 16th century by Blaise de Vigenere. The Vigenere cipher uses a 26×26 table with A to Z as the row heading and column heading. This table is usually referred to as the Vigenere Tableau, Vigenere Table or Vigenere Square. The first row of this table has the 26 English letters. Starting with the second row, each row has the letters shifted to the left one position in a cyclic way. For example, when B is shifted to the first position on the second row, the letter A moves to the end.

Encryption:

- Find the row corresponding to the current letter of the keyword.
- Find the column corresponding to the current letter of the plaintext.
- The intersection of this row and column gives the encrypted letter.

Decryption:

- Find the row corresponding to the current letter of the keyword.
- Search this row to find the column that matches the encrypted letter.
- The header of this column gives the decrypted letter.

Source Code:

```
#include<stdio.h>
#include<string.h>
#include<conio.h>

int main()
{
    char msg[200],key[200];
    int msgLen,keyLen,i,j;
    char newKey[200],encryptedMsg[200],decryptedMsg[200];
    clrscr();
    printf("Input message:");
    gets(msg);
    printf("Input key:");
    gets(key);
    msgLen=strlen(msg);
    keyLen=strlen(key);
    //generating new key
    for(i=0,j=0;i<msgLen;++i,++j)
    {
        if(j==keyLen)
            j=0;
        newKey[i]=key[j];
    }
    newKey[i] = '\0';
    //encryption
    for(i=0;i<msgLen;++i)
        encryptedMsg[i]=((msg[i]+newKey[i])%26)+'A';
    encryptedMsg[i]='\0';
    //decryption
    for(i=0;i<msgLen;++i)
        decryptedMsg[i]=(((encryptedMsg[i]-newKey[i])+26)%26)+'A';
    decryptedMsg[i]='\0';
    printf("Original Message:%s",msg);
    printf("\nKey:%s",key);
    printf("\nNew Generated Key: %s",newKey);
    printf("\nEncrypted Message: %s",encryptedMsg);
    printf("\nDecrypted Message: %s",decryptedMsg);
    getch();
    return 0;
}
```

Output:

```
Select H:\BCA\8th Semester\Lab Reports\IS Lab C\VIGENERE.exe
Input message:MECHI
Input key:MMC
Original Message:MECHI
Key:MMC
New Generated Key: MMCMM
Encrypted Message: YQETU
Decrypted Message: MECHI_
```

Lab 3: Implementation of Playfair Cipher

Introduction:

The Playfair cipher is a manual symmetric encryption technique and was the first digraph substitution cipher. The scheme was invented in 1854 by Charles Wheatstone but bears the name of Lord Playfair for promoting its use. How the Playfair Cipher Works:

1. Key Table Preparation:
 - Choose a keyword or phrase.
 - Write the keyword (removing duplicates) followed by the remaining letters of the alphabet that are not in the keyword.
 - Typically, 'I' and 'J' are combined into one slot to fit the 25-letter grid (5x5). Arrange these letters in a 5x5 grid.
2. Formatting the Plaintext:
 - If the plaintext has an odd number of characters, add an extra letter (commonly 'Z') to make it even.
 - Divide the text into pairs of two letters. If both letters in the pair are the same, insert a filler letter, such as 'Z', between them.
3. Encrypting Each Pair:
 - Same Row: If the pair is in the same row, replace them with the letters to their immediate right respectively (wrap around to the beginning of the row if necessary).
 - Same Column: If the pair is in the same column, replace them with the letters immediately below respectively (wrap around to the top if necessary).
 - Rectangle Rule: If the letters are not on the same row or column, form a rectangle with the two letters and take letters on the same row respectively but at the opposite corners of the rectangle.
4. Decrypting the Cipher text: The decryption process is the reverse of the encryption process using the same key.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <conio.h>

#define SIZE 30

// Function to convert the string to lowercase
void toLowerCase(char plain[], int ps)
{
    int i;
    for (i = 0; i < ps; i++) {
        if (plain[i] > 64 && plain[i] < 91)
            plain[i] += 32;
    }
}

// Function to remove all spaces in a string
int removeSpaces(char* plain, int ps)
{
    int i, count = 0;
    for (i = 0; i < ps; i++)
        if (plain[i] != ' ')
            plain[count++] = plain[i];
    plain[count] = '\0';
    return count;
}

// Function to generate the 5x5 key square
void generateKeyTable(char key[], int ks, char keyT[5][5])
{
    int i, j, k, flag = 0, *dicty;

    // a 26 character hashmap
    // to store count of the alphabet
    dicty = (int*)calloc(26, sizeof(int));
    for (i = 0; i < ks; i++) {
        if (key[i] != 'j')
            dicty[key[i] - 97] = 2;
    }

    dicty['j' - 97] = 1;
```

```

i = 0;
j = 0;

for (k = 0; k < ks; k++) {
    if (dicty[key[k] - 97] == 2) {
        dicty[key[k] - 97] -= 1;
        keyT[i][j] = key[k];
j++;
        if (j == 5) {
            i++;
            j = 0;
        }
    }
}

for (k = 0; k < 26; k++) {
    if (dicty[k] == 0) {
        keyT[i][j] = (char)(k + 97);
        j++;
        if (j == 5) {
            i++;
            j = 0;
        }
    }
}

// Function to search for the characters of a digraph
// in the key square and return their position
void search(char keyT[5][5], char a, char b, int arr[])
{
    int i, j;

    if (a == 'j')
        a = 'i';
    else if (b == 'j')
        b = 'i';

    for (i = 0; i < 5; i++) {

        for (j = 0; j < 5; j++) {

```

```

        if (keyT[i][j] == a) {
            arr[0] = i;
            arr[1] = j;
        }

        else if (keyT[i][j] == b) {
            arr[2] = i;
            arr[3] = j;
        }
    }
}
}

```

```

// Function to find the modulus with 5
int mod5(int a) { return (a % 5); }

```

```

// Function to make the plain text length to be even
int prepare(char str[], int ptrs)
{
    if (ptrs % 2 != 0) {
        str[ptrs++] = 'z';
        str[ptrs] = '\0';
    }
    return ptrs;
}

```

```

// Function for performing the encryption
void encrypt(char str[], char keyT[5][5], int ps)
{
    int i, a[4];

    for (i = 0; i < ps; i += 2) {

        search(keyT, str[i], str[i + 1], a);

        if (a[0] == a[2]) {
            str[i] = keyT[a[0]][mod5(a[1] + 1)];
            str[i + 1] = keyT[a[0]][mod5(a[3] + 1)];
        }
        else if (a[1] == a[3]) {
            str[i] = keyT[mod5(a[0] + 1)][a[1]];
            str[i + 1] = keyT[mod5(a[2] + 1)][a[1]];
        }
        else {

```

```

        str[i] = keyT[a[0]][a[3]];
        str[i + 1] = keyT[a[2]][a[1]];
    }
}

// Function to encrypt using Playfair Cipher
void encryptByPlayfairCipher(char str[], char key[])
{
    char ps, ks, keyT[5][5];
    // Key
    ks = strlen(key);
    ks = removeSpaces(key, ks);
    toLowerCase(key, ks);
    // strlwr(key);
    // Plaintext
    ps = strlen(str);
    toLowerCase(str, ps);
    ps = removeSpaces(str, ps);

    ps = prepare(str, ps);

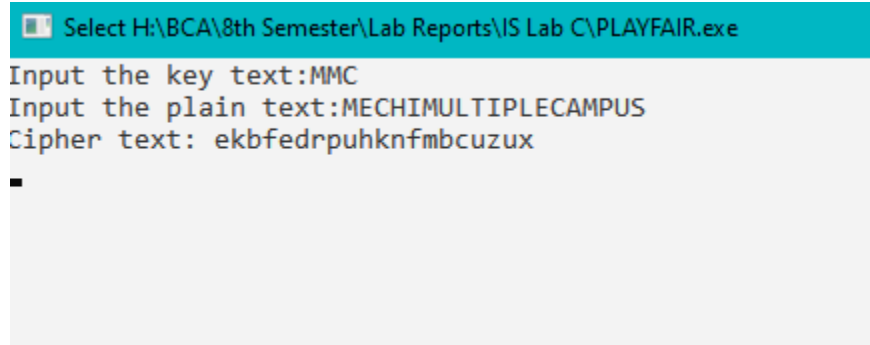
    generateKeyTable(key, ks, keyT);

    encrypt(str, keyT, ps);
}

int main()
{
    char str[SIZE], key[SIZE];
    clrscr();
    // Key to be encrypted
    printf("Input the key text:");
    gets(key);
    // Plaintext to be encrypted
    printf("Input the plain text:");
    gets(str);
    // encrypt using Playfair Cipher
    encryptByPlayfairCipher(str, key);
    printf("Cipher text: %s\n", str);
    getch();
    return 0;
}

```

Output:



```
Select H:\BCA\8th Semester\Lab Reports\IS Lab C\PLAYFAIR.exe
Input the key text:MMC
Input the plain text:MECHIMULTIPLECAMPUS
Cipher text: ekbfedrpuhknfmbcuzux
```

Lab 4: Rail Fence Ciphers

Introduction:

The Rail Fence Cipher is a form of transposition cipher that rearranges the letters of the plaintext without altering the actual letters themselves. It is called a "rail fence" because it writes the message in a zigzag pattern over a number of rows (rails), and then reads off each row one by one. How Rail Fence Cipher Works:

- Choose the Number of Rails: Decide how many levels (rows) the text will be split across.
- Write Down in a Zigzag: Write the plaintext in a diagonal, zigzag pattern across the specified number of rails.
- Read Off Horizontally: Once written, the message is then read off in rows.

Source Code:

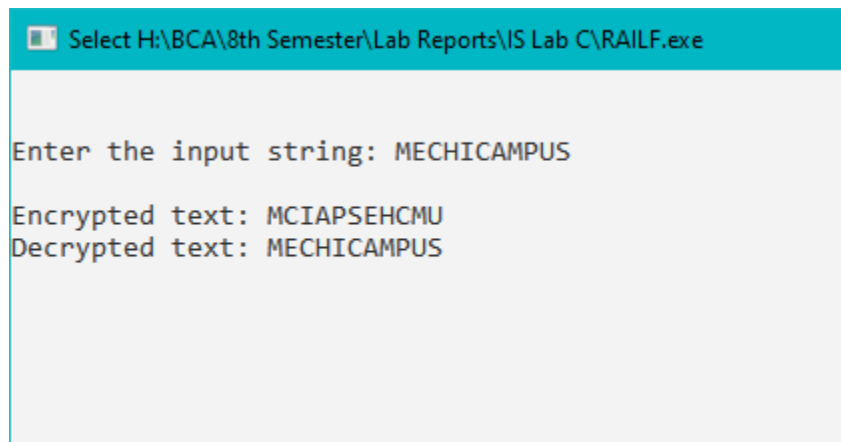
```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    int i,j,k,l;
    char a[20],c[20],d[20];
    clrscr();
    printf("\n\nEnter the input string : ");
    gets(a);
    l=strlen(a);
    /*Encrypting*/
    for(i=0,j=0;i<l;i++)
    {
        if(i%2==0)
            c[j++]=a[i];
    }
    for(i=0;i<l;i++)
    {
```

```

        if(i%2==1)
            c[j++]=a[i];
    }
    c[j]='\0';
    printf("\nEncrypted text:");
    printf("%s",c);
    /*Decrypting*/
    if(l%2==0)
        k=l/2;
    else
        k=(l/2)+1;
    for(i=0,j=0;i<k;i++)
    {
        d[j]=c[i];
        j=j+2;
    }
    for(i=k,j=1;i<l;i++)
    {
        d[j]=c[i];
        j=j+2;
    }
    d[l]='\0';
    printf("\nDecryption Text: ");
    printf("%s",d);
    getch();
}

```

Output:



```

Select H:\BCA\8th Semester\Lab Reports\IS Lab C\RAILF.exe

Enter the input string: MECHICAMPUS

Encrypted text: MCIAPSEHCMU
Decrypted text: MECHICAMPUS

```

Lab 5: Implementation of password based authentication system

Introduction:

Password-based authentication system is a security mechanism used to restrict unauthorized access to systems, applications, or data. Users must enter a predefined password that matches the one stored in the system to gain access. In this lab, we will implement a simple password-based authentication system using C programming. The system will prompt the user to enter a password and compare it to a stored password. If the passwords match, the user is granted access; otherwise, access is denied.

Source Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_PASSWORD_LENGTH 20

int main() {
    char storedPassword[MAX_PASSWORD_LENGTH] = "password123"; // predefined password
    char inputPassword[MAX_PASSWORD_LENGTH];
    int attempts = 3; // Number of allowed attempts

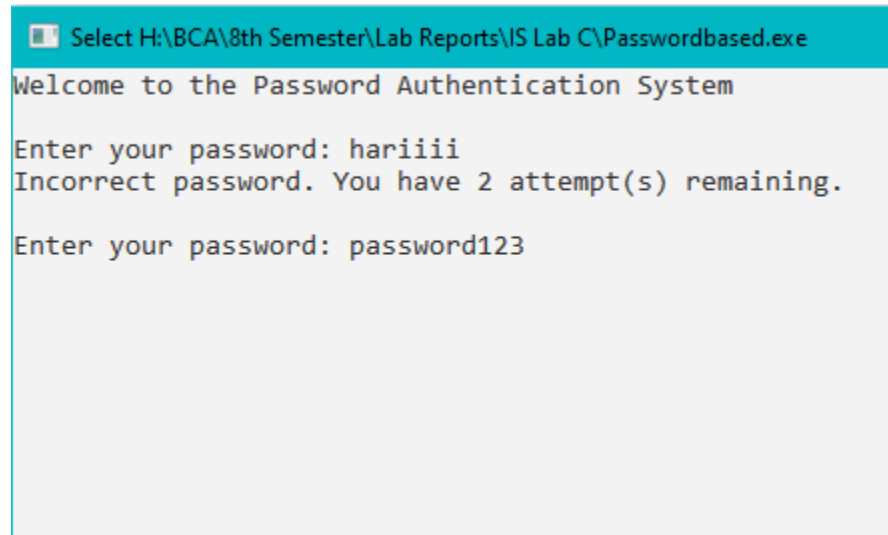
    printf("Welcome to the Password Authentication System\n");

    // Loop for 3 attempts
    while (attempts > 0) {
        printf("\nEnter your password: ");
        scanf("%s", inputPassword);

        // Compare entered password with the stored password
        if (strcmp(storedPassword, inputPassword) == 0) {
            printf("Access Granted!\n");
            return 0; // Exit the program successfully
        } else {
            attempts--;
            if (attempts > 0) {
                printf("Incorrect password. You have %d attempt(s) remaining.\n", attempts);
            } else {
                printf("Access Denied. No more attempts remaining.\n");
            }
        }
    }
}
```

```
    return 0; // Exit the program after failed attempts  
}
```

Output:



The screenshot shows a Windows command prompt window with a title bar that reads "Select H:\BCA\8th Semester\Lab Reports\IS Lab C\Passwordbased.exe". The window contains the following text:

```
Welcome to the Password Authentication System  
  
Enter your password: hariiii  
Incorrect password. You have 2 attempt(s) remaining.  
  
Enter your password: password123
```